

# Arquitectura de Computadoras

Parcial de práctica 2021

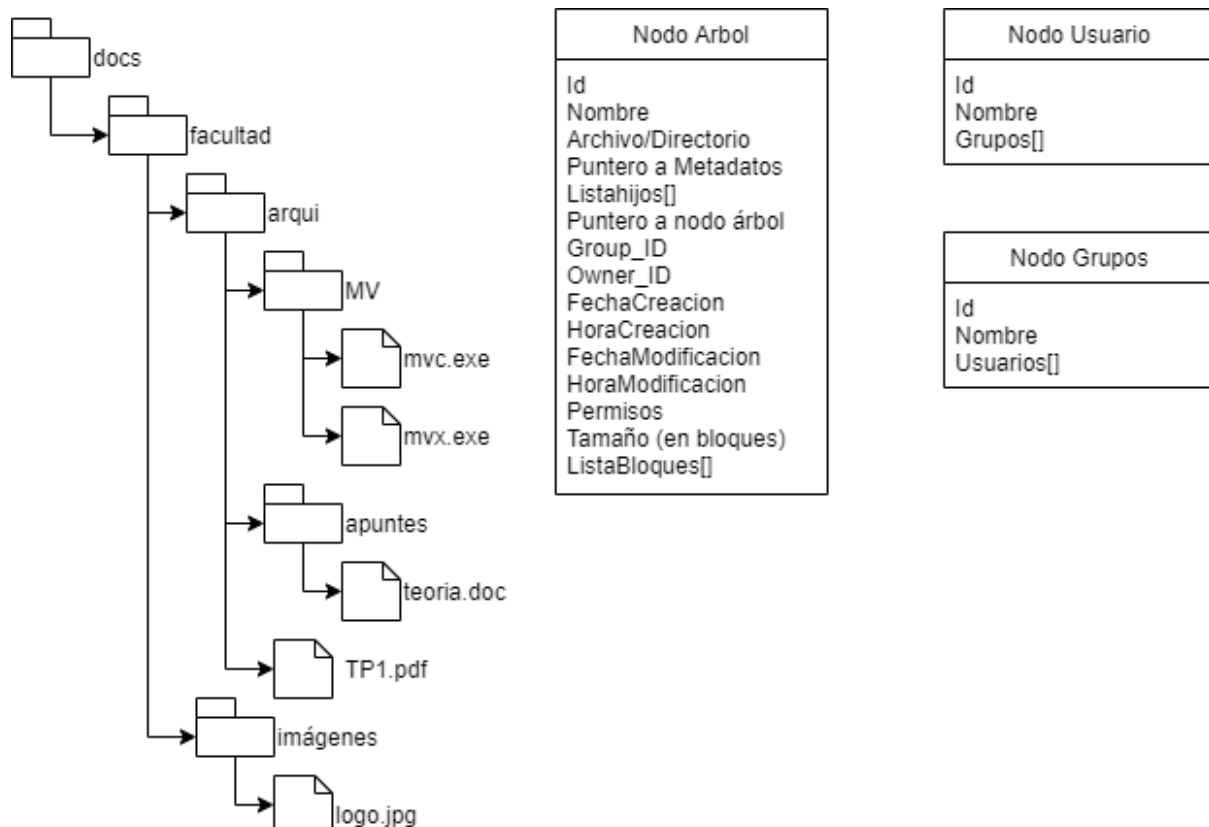
## Visión general

Un sistema de archivos es representado por un árbol. Cada nodo del árbol puede ser un archivo o un directorio (carpeta). Los nodos directorios pueden tener hijos que pueden ser archivos o directorios.

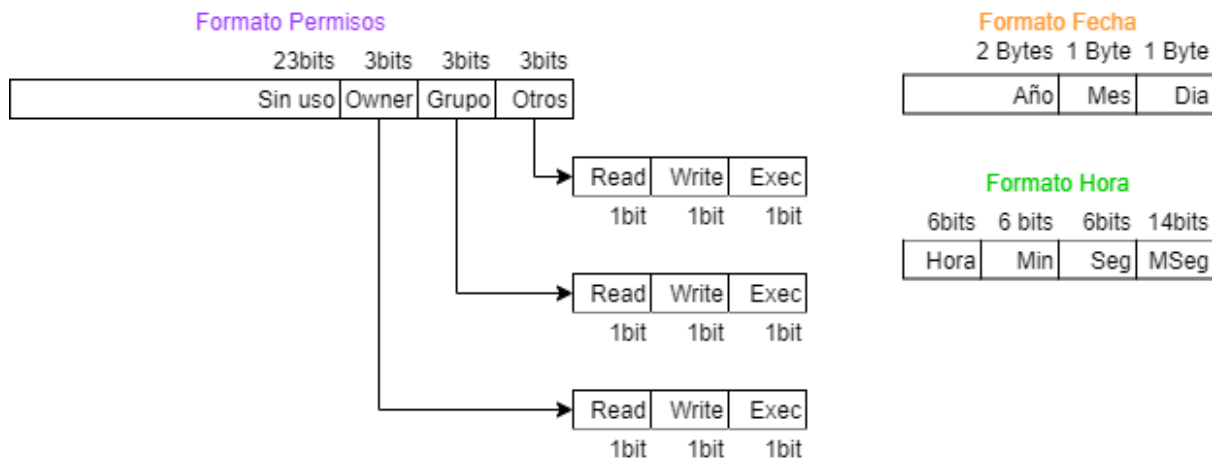
Dentro de cada nodo del árbol se encuentran los metadatos del archivo/directorio (Nombre, fecha de creación, fecha de modificación, tamaño, etc).

Cada archivo o directorio está asociado a un usuario llamado **owner** (el dueño del archivo) y a un grupo (**group**) al que pertenece el archivo.

Además el sistema de archivos tiene una (o varias) estructura(s) de datos donde se organizan los usuarios y los grupos. Cada usuario pertenece a 1 o más grupos. La información es coherente y simétrica, si un usuario pertenece a un grupo, ese grupo contendrá al usuario. No puede haber 2 usuarios con el mismo nombre, ni 2 grupos con el mismo nombre.



# Tipos de Datos



**Formato de fecha:** 1 Celda de 4 bytes, en los cuales son, de más significativos a menos significativos: 2 bytes para el año, 1 byte para el mes (1 a 12) y 1 byte para el día (1 a 30).

**Formato de hora:** 1 celda de 32 bits, en los cuales son, de más significativo a menos significativos: 6 bits para la hora (0 a 23), 6 bits para los minutos (de 0 a 59), 6 bits los segundos (0 a 59) y 14 bits para milésimas de segundos (0 a 999).

## Formato de Permisos:

Representan el acceso que puede tener un usuario a un archivo o directorio. Es un atributo propio del archivo o directorio (carpeta). Consta de 1 celda donde solo se utilizan los 9 bits menos significativos.

De más significativo a menos significativo son: 3 bits para el **owner**, 3 para **grupo** y 3 bits para **otros** usuarios.

A su vez cada permiso se subdivide en 1 bit para acceso de lectura, 1 bit para acceso escritura y 1 bit para acceso de ejecución.

Cuando el bit tiene valor 1 significa que el permiso está otorgado, cuando está en 0 el permiso está denegado.

Se recomienda el uso de sistema de numeración octal para trabajar con permisos.

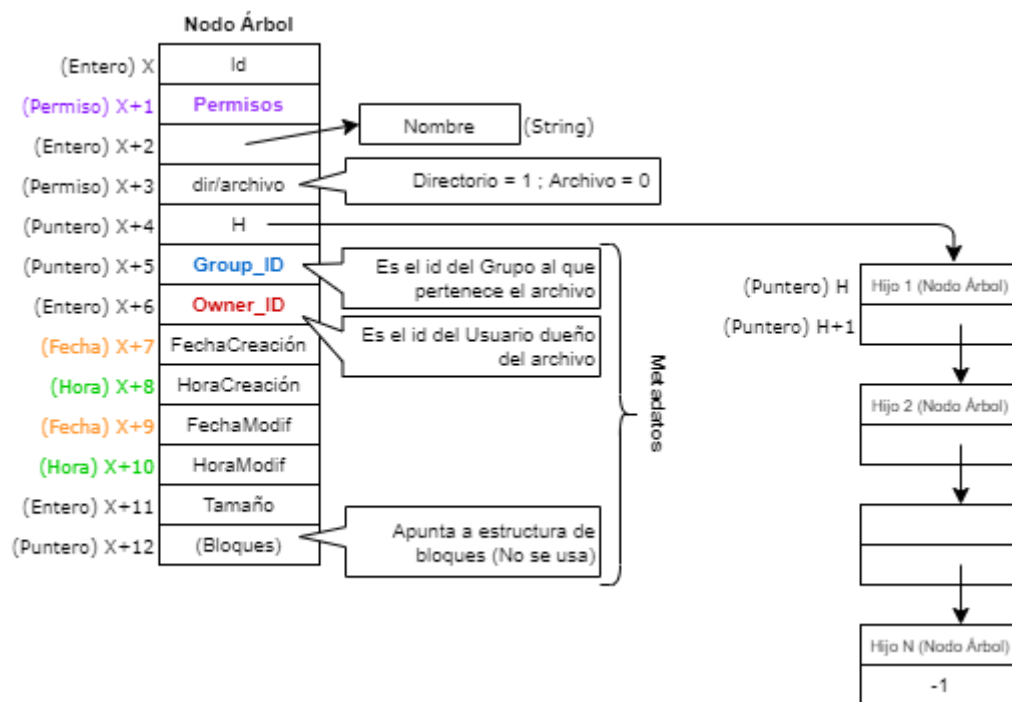
Ejemplos:

- Si en permisos hay un **@777** (todos los bits en 1) quiere decir que tanto el owner, como el miembro del grupo, como cualquier usuario tiene permiso de lectura escritura y ejecución.
- Si en permisos hay un **@751** (binario 111 101 001) quiere decir que el owner puede leer, escribir y ejecutar; el miembro del grupo puede leer y ejecutar pero no escribir; y por último el resto de los usuarios pueden solo ejecutar.

Para validar si un usuario tiene permisos se debe seguir el siguiente criterio:

- 1) Verificar si es el owner del archivo, y luego mirar solo los permisos del owner (**@700**)
- 2) Verificar si es miembro del grupo, y luego mirar solo los permisos del grupo (**@070**)
- 3) Si no entró ninguno de los casos anterior, mirar los permisos de otros (**@007**)

# Árbol



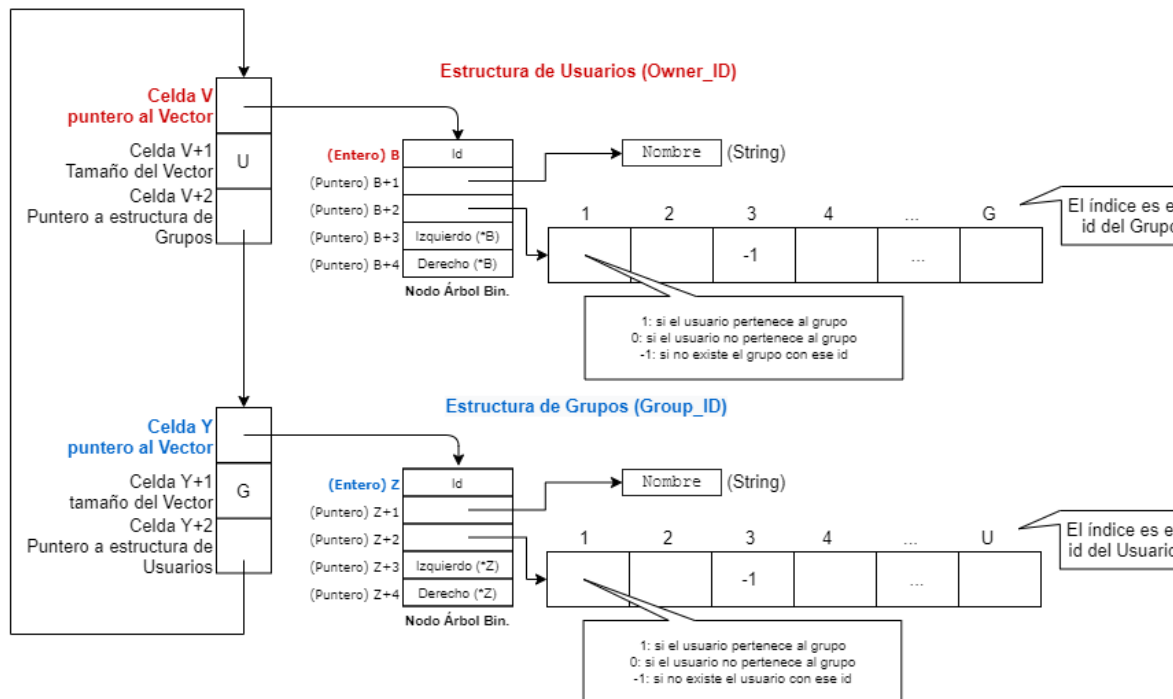
Los archivos y directorios (carpetas) se representan con un nodo de un árbol en una dirección X de memoria. Se diferencia archivos de directorios por el campo en X+2 que si tiene un **1** el nodo corresponde a un **directorio** y si tiene un **0** corresponde a un **archivo**.

Cada nodo del árbol también contiene los metadatos de los archivos/directorios. Ahí mismo se encuentra el **id del usuario** (entero positivo) que es dueño del archivo (OWNER\_ID) en X+6 y el **id del grupo** (entero positivo) al que pertenece el archivo (GROUP\_ID) en X+5.

Los campos tamaño (X+11) y bloques (X+12) **solo son válidos para los archivos**.

**Si** el nodo **es un directorio**, en X+4 **apunta a una lista de hijos**. Cada nodo de la lista tiene un puntero a nodo de árbol y un puntero al siguiente. **Si** el directorio **está vacío** en X+4 habrá un -1 (null).

# Usuarios y grupos



La estructura de **usuarios** se compone de 3 celdas con:

- V apunta al nodo raíz de un árbol binario de búsqueda ordenado por el id del usuario. Donde cada nodo se compone de 5 celdas:
  - B contiene el id del usuario.
  - B+1 contiene un puntero a un string con el nombre del usuario.
  - B+2 apunta a un vector donde el índice de la celda corresponde al id de un grupo y el contenido puede ser 1 si el usuario pertenece al grupo, 0 si el usuario no pertenece o -1 si el grupo no existe.
- V+2 contiene la cantidad de usuarios.
- V+3 contiene un puntero a la estructura de grupos Y

La estructura de **grupos**, es idéntica a la de usuarios: El árbol binario está ordenado por el id del grupo, y el índice del vector de usuarios es el id de usuario conteniendo 1, 0 o -1 en los casos que el usuario pertenezca al grupo, no pertenezca o no exista.

Tanto en **usuarios** como en **grupos** si la celda que apunta al nombre tiene -1 (NULL), el usuario o grupo está eliminado (es decir que ese id está disponible para ser utilizado).

Entonces, para eliminar un usuario o un grupo, no se debe eliminar el nodo, solo marcarlo con NULL sobre el nombre.

## Ejercicio

Escriba la (o las) subrutina(s) necesaria(s) para resolver el siguiente problema:

**Devolver en una lista todos los id y nombre de los archivos, que han sido modificados luego de una fecha y hora dada, y que poseen permiso de ejecución de un usuario determinado,** La subrutina debe buscar recursivamente en todo el sistema de archivos. Se debe especificar la estructura de la lista.

La subrutina principal tiene como parámetros:

- un puntero a un directorio (X) raíz del sistema de archivos,
- la fecha,
- la hora,
- el id de usuario,
- la dirección de memoria (V) con a la estructura de usuarios del sistema,
- la dirección de memoria donde se encuentra el puntero al primer nodo de la lista.

Y debe responder al siguiente protocolo en C:

```
void list (TArbol* raiz, TFecha fecha, THora hora, int idUsuario, TUsuarios* usrs, Tlist* lista)
```

**Nota:** debe contemplar solicitar memoria si es necesario.

## Solución

```
PUSH <puntero a la cabecera de la lista>
PUSH <puntero a la estructura de usuarios> ; V
PUSH <id del usuario>
PUSH <hora>
PUSH <fecha>
PUSH <puntero a la raíz del sistema de archivos> ; X
CALL list
ADD SP, 6
; la cabecera de la lista debe estar inicializada con null

PUSH <hora>
PUSH <fecha>
PUSH <puntero al archivo> ; X
CALL modified
ADD SP, 3
; devuelve en AX si el archivo fue modificado luego de la fecha y hora

PUSH <puntero a la estructura de usuarios> ; V
PUSH <id del usuario>
PUSH <puntero al archivo> ; X
CALL execute
ADD SP, 3
; devuelve en AX si el usuario tiene permiso de ejecución sobre el archivo

PUSH <puntero al nodo de un usuario> ; B
```

```

PUSH <id del usuario>
PUSH <id del grupo>
CALL user_group
ADD SP, 3
; devuelve en FX si el usuario forma parte del grupo

```

```

PUSH <puntero a la cabecera de la lista>
PUSH <puntero al archivo> ; X
CALL add_file
ADD SP, 2

```

```

; Estructura de un nodo de la lista:
; L   | id del archivo
; L+1 | puntero al nombre
; L+2 | puntero al siguiente

```

```

true    EQU 1
false   EQU 0
null    EQU -1

```

```

list_list    EQU 7
list_users   EQU 6
list_us_id   EQU 5
list_hour    EQU 4
list_date    EQU 3
list_root    EQU 2

```

```

list:        PUSH BP
              MOV BP, SP
              PUSH AX
              PUSH BX

              MOV BX, [BP+list_root] ; BX = X

              CMP [BX+3], 0
              JZ is_file

              ; es un directorio
              MOV BX, [BX+4] ; BX = H

childs:      CMP BX, null
              JZ end_list

```

```

              PUSH [BP+list_list]
              PUSH [BP+list_users]
              PUSH [BP+list_us_id]
              PUSH [BP+list_hour]
              PUSH [BP+list_date]

```

```
PUSH [BX] ; puntero al hijo
CALL list
ADD SP, 6
```

```
MOV BX, [BX+1] ; BX = [H+1]
```

```
JMP childs
```

```
is_file:  PUSH [BP+list_hour]
          PUSH [BP+list_date]
          PUSH BX ; puntero al archivo
          CALL modified
          ADD SP, 3
```

```
CMP AX, false
JZ end_list ; no fue modificado luego de esa fecha y hora
```

```
; sí fue modificado luego de esa fecha y hora
PUSH [BP+list_users]
PUSH [BP+list_us_id]
PUSH BX ; puntero al archivo
CALL execute
ADD SP, 3
```

```
CMP AX, false
JZ end_list ; el usuario no tiene permiso de ejecución
```

```
; el usuario sí tiene permiso de ejecución
PUSH [BP+list_list]
PUSH BX ; puntero al archivo
CALL add_file
ADD SP, 2
```

```
end_list: POP BX
          POP AX
          MOV SP, BP
          POP BP
          RET
```

```
modif_hour EQU 4
modif_date EQU 3
modif_file EQU 2
```

```
modified:  PUSH BP
          MOV BP, SP
          PUSH BX
```

```
MOV AX, false
```

```
MOV BX, [BP+modif_file]
```

```
CMP [BX+9], [BP+modif_date] ; comparo las fechas
JP is_modif ; fue modificado luego de esa fecha
JN end_modif ; no fue modificado luego de esa fecha
```

```
; fue modificado en esa misma fecha
CMP [BX+10], [BP+modif_hour] ; comparo las horas
JNP end_modif ; no fue modificado luego de esa hora
```

```
; fue modificado luego de esa hora
```

```
is_modif: MOV AX, true
```

```
end_modif: POP BX
MOV SP, BP
POP BP
RET
```

```
exec_users EQU 4
exec_us_id EQU 3
exec_file EQU 2
```

```
execute: PUSH BP
MOV BP, SP
PUSH BX
PUSH CX
PUSH DX
PUSH FX
```

```
MOV AX, false
```

```
MOV BX, [BP+exec_file] ; BX = X
```

```
MOV CX, [BX+1] ; CX = permisos
```

```
CMP [BX+6], [BP+exec_us_id] ; comparo el id del owner con el del usuario
JZ is_owner
```

```
; no es owner
```

```
MOV DX, [BP+exec_users] ; DX = V
```

```
MOV EX, [DX+2] ; EX = Y
```

```
CMP [EX+1], [BX+5] ; comparo el tamaño del vector de grupos con el Group_id
JN is_other ; el Group_id es mayor que el tamaño del vector
```

```
; el Group_id es menor o igual que el tamaño del vector
```

```
PUSH [DX] ; B
PUSH [BP+exec_us_id]
PUSH [BX+5] ; Group_id
CALL user_group
```



```

        ADD SP, 3

        CMP FX, true
        JZ is_group

        ; no está en el grupo
        ; es "otros"
is_other: AND CX, @001
        JZ end_exec
        JMP has_exec

is_group: AND CX, @010
        JZ end_exec
        JMP has_exec

is_owner: AND CX, @100
        JZ end_exec
        JMP has_exec

has_exec: MOV AX, true

end_exec: POP FX
        POP DX
        POP CX
        POP BX
        MOV SP, BP
        POP BP
        RET


usgr_node EQU 4
usgr_us_id EQU 3
usgr_gr_id EQU 2

user_group: PUSH BP
        MOV BP, SP
        PUSH BX

        MOV BX, [BP+usgr_node] ; BX = B
        CMP BX, NULL
        JZ not_member ; llegué al final del árbol y no encontré el usuario

        CMP [BX+0], [BP+usgr_us_id] ; comparo el id del nodo con el del usuario
        JP usgr_izq ; el id del nodo es mayor que el del usuario
        JN usgr_der ; el id del nodo es menor que el del usuario

        ; el nodo es el del usuario buscado
        MOV BX, [BX+2]
        ADD BX, [BP+usgr_gr_id]
        SUB BX, 1
        ; BX queda en la posición del grupo en el vector de grupos del usuario

```

```
CMP [BX], 1
JNZ not_member ; el usuario no pertenece al grupo o el grupo no existe
MOV FX, true ; el usuario sí pertenece al grupo
JMP fin_usgr
```

```
usgr_izq:  PUSH [BX+3]
           PUSH [BP+usgr_us_id]
           PUSH [BP+usgr_gr_id]
           CALL user_group
           ADD SP, 3
           JMP fin_usgr
```

```
usgr_der:  PUSH [BX+4]
           PUSH [BP+usgr_us_id]
           PUSH [BP+usgr_gr_id]
           CALL user_group
           ADD SP, 3
           JMP fin_usgr
```

```
not_member: MOV FX, false
```

```
fin_usgr:  POP BX
           MOV SP, BP
           POP BP
           RET
```

```
addf_list EQU 3
addf_file EQU 2
```

```
add_file:  PUSH BP
           MOV BP, SP
           PUSH BX
           PUSH CX
           PUSH DX
           PUSH EX
```

```
MOV BX, [BP+addf_file] ; BX = X
MOV EX, [BP+addf_list] ; EX = puntero a la cabecera
```

```
MOV CX, 3
SYS 5
```

```
; DX = puntero al nuevo nodo
CMP DX, null
JZ end_addf ; no hay memoria disponible
```

```
MOV [DX+0], [BX+0] ; copio el id
MOV [DX+1], [BX+2] ; copio el puntero al nombre
MOV [DX+2], [EX] ; el siguiente es el primero de la lista
```

```
MOV [EX], DX ; el nuevo nodo es el nuevo primero
```

```
end_addf: POP EX  
          POP DX  
          POP CX  
          POP BX  
          MOV SP, BP  
          POP BP  
          RET
```